

Project Interim Report On

Online Voting System

(Using Core Java & OOP Concepts)

RU-100-01-00012
Object Oriented Programming With Java

Session of 2025-26



Submitted by
Tasneem Hashmi
RU-25-11528

**Bachelor of Technology in Computer Science &
Engineering in Association With Google**

Under the guidance of
Mr. Harsh Multani
Java Trainer

School of Computer Science and Engineering
Rungta International Skills University
Bhilai, Chhattisgarh

(April 2026)

Certificate

This is to certify that Tasneem Hashmi, bearing ERP Id. RU-25-11528, a student of Bachelor of Technology in Computer Science & Engineering in Association with Google, has successfully completed a student project entitled “**Online Voting System**” as a part of the Object Oriented Programming With Java course requirements for the session 2025-26.

The project is a genuine and legitimate work of Tasneem Hashmi and was conducted from 11 April 2026 until , under the supervision and guidance of Mr. Harsh Multani, Java Trainer.

Acknowledgment

I would like to express my sincere gratitude to my project guide and Java trainer, Mr. Harsh Multani, for his valuable guidance, support and encouragement throughout the course of this project. Mr. Multani's expertise and insightful suggestions greatly contributed to the successful completion of this work. The project helped me understand and implement various approaches and concepts of Object Oriented Programming using Java.

I would also like to extend my gratitude to the Head of Department, Dr. Anurag Sharma, and all the faculty members of the Computer Science & Engineering Department, for providing the necessary resources and academic environment to carry out this project.

Finally, I would like to thank my family for their constant support, understanding and encouragement, which helped me complete this project successfully.

Tasneem Hashmi
RU-25-11528

Table of Contents

01 Abstract	5
--------------------	---

02 Introduction	6
------------------------	---

03 Scope	
-----------------	--

04 System Requirements	
-------------------------------	--

05 Methodology	
-----------------------	--

06 Implementation	
--------------------------	--

07 Gantt Chart	
-----------------------	--

08 Conclusion	
----------------------	--

09 References	
----------------------	--

Abstract

The Online Voting System is a console-based Java application designed to simulate a secure and efficient voting process. The system allows users (voters) to cast votes for candidates, ensures that each voter can vote only once, and calculates the final results by counting votes.

This project uses Object Oriented Programming fundamentals and concepts like encapsulation, class design, and method implementation to interact with the user (voter) and seek their voter ID and candidate choice to check whether the user is eligible to vote, has voted or not to prevent duplicate voting by maintaining a collection (ArrayList and/or HashSet) to store voter ID. When a voter inputs his voter ID, the collection is searched through to determine whether the voter has voted. If not, program runs further by allowing the voter to vote for his candidate of choice. The vote count for the chosen candidate is incremented and stored in a collection of candidates-to-vote count mapping. The final result is then displayed. The system uses file handling to store the voting data efficiently for further analysis and to ensure a permanent record is maintained, thereby supporting real-time data integration.

(This project is currently under development and, additional features and enhancements will be incorporated in future iterations.)

Introduction

Voting is one of the most important mechanisms for collecting opinions, making decisions, and selecting representatives in democratic and organisational environments. Traditional voting methods often require manual record keeping and vote counting, which can be time-consuming and prone to human error. A programmed Voting System helps automate every voting session for each user, increasing security and efficiency.

The Online Voting System is a console-based application developed using Java that simulates a simple electronic voting process. The primary objective of the system is to allow eligible voters to cast their votes for a candidate while ensuring that each voter is allowed to vote only once. The system records votes, maintains voter information, and automatically calculates the election results based on the votes received by each candidate.

This project has been developed as part of the Object Oriented Programming with Java course to demonstrate the practical implementation of OOP principles in a real-world scenario. The application makes use of classes, objects, encapsulation, methods, collections, and conditional statements to create a structured and maintainable software solution. By representing candidates as objects and managing voter records through Java collections, the system achieves efficient vote management and duplicate vote prevention.

The Online Voting System consists of several key functionalities. It provides a menu-driven interface through which users can cast votes, view election results, and exit the application. Before accepting a vote, the system verifies the voter's unique identification number to ensure that the voter has not already participated in the election. If the voter is eligible, the selected candidate's vote count is updated accordingly. The system then stores the voter ID to prevent any future duplicate voting attempts.

The project also demonstrates the practical use of Java Collection Framework classes such as ArrayList and HashSet. The ArrayList is used to store and manage candidate information, while the HashSet efficiently maintains the list of voter IDs that have already voted. This approach provides fast lookup operations and helps maintain the integrity of the voting process. Apart from implementing core Java concepts, the project encourages students to understand the software development lifecycle, including requirement analysis, system design, implementation, testing, and documentation. The application serves as a simplified model of real-world electronic voting systems and provides a strong foundation for future enhancements such as graphical user interfaces, database integration, online connectivity, administrative controls, secure authentication mechanisms, and persistent data storage.

Overall, the Online Voting System is an educational project that combines programming logic with Object-Oriented Programming principles to create a practical and user-friendly application. It demonstrates how Java can be used to solve real-world problems through efficient data management, validation techniques, and structured program design.

Scope

This system is intended for small-scale applications such as classroom voting, organisational polls, or simply for learning purposes.

The current version is a console-based application and does not include advanced features such as database connectivity, user authentication, online access, or permanent data storage. However, the system can be extended in the future by incorporating graphical interfaces, secure login mechanisms, database integration, and support for multiple elections.

System Requirements

Hardware: Computer System with a minimum of 4GB RAM

Software: Java JDK, VSCode Terminal, Git

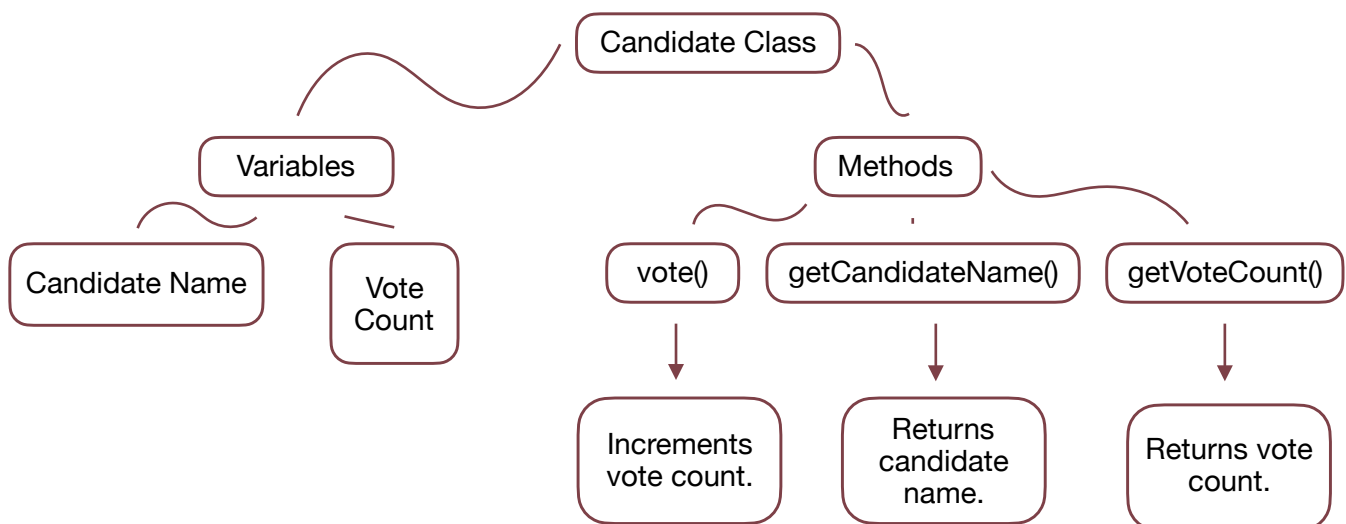
Methodology

1. Requirement Analysis

The requirements of the Online Voting System were identified before development. The system was designed to:

- Allow users to cast votes for candidates.
- Prevent duplicate voting.
- Count votes automatically.
- Display election results accurately.
- Provide a simple menu-driven interface.

2. System Design



Data Structures used:

HashSet of Java Collection
ArrayList of Java Collection

3. Voting Process

The voting process works like:

- User selects the **Vote** option.
- System asks for the Voter ID.
- Voter ID is checked in the HashSet.
- If the ID already exists:
 - Voting is rejected.
 - User is informed that they have already voted.

If the ID is valid:

- Candidate list is displayed.

- User selects a candidate.

- Vote count is incremented.

- Voter ID is stored in the HashSet.

4. Result Generation

After voting is completed:

- Vote count of each candidate is displayed.

- Candidate with the highest vote count is identified.

- Winner is declared automatically.

Implementation

The Online Voting System was implemented using Java and follows an Object-Oriented Programming approach. The application is console-based and consists of a Candidate class and a main VotingSystem class that manages the voting process.

1. Candidate Class

The Candidate class is used to represent a candidate participating in the election. It contains the candidate's name and vote count as private data members, ensuring encapsulation.

Attributes

- candidateName : Stores the name of the candidate.
- voteCount : Stores the total number of votes received.

Methods

- vote() : Increments the vote count by one.
- getCandidateName() : Returns the candidate's name.
- getVoteCount() : Returns the current vote count.

The class allows candidate information to be managed efficiently while restricting direct access to internal data.

2. Candidate Management

An ArrayList is used to store candidate objects dynamically. During program initialization, candidate objects are created and added to the list. This approach allows easy retrieval and display of candidates during the voting process.

Example Candidates:

Tasneem
Sebastian
Harshita

3. Voter Management

A HashSet is used to maintain voter IDs. Since a HashSet stores only unique values, it is ideal for preventing duplicate voting.

Before accepting a vote, the system checks whether the entered voter ID already exists in the HashSet:

- If the voter ID exists, the vote is rejected.
- If the voter ID does not exist, voting is allowed and the voter ID is stored after the vote is cast.

4. Voting Process

The voting functionality is implemented through the castVote() method.

The process includes:

- Accepting the voter ID.
- Checking for duplicate voting.
- Displaying the list of candidates.
- Accepting the candidate choice.
- Validating the selected option.
- Recording the vote.
- Storing the voter ID.

This ensures that only valid votes are counted and duplicate voting attempts are prevented.

5. Result Processing

The `displayResults()` method is responsible for displaying election results.

The method:

- Displays the vote count of each candidate.
- Compares vote counts.
- Identifies the candidate with the highest number of votes.
- Declares the winner.

The result generation process is automatic and does not require manual vote counting.

6. User Interface

The application uses a menu-driven interface implemented through a loop and switch statement.

Menu Options:

1. Vote
2. View Result
3. Exit

The menu continues to execute until the user chooses the Exit option, providing a simple and user-friendly interaction mechanism.

7. OOP Concepts Implemented

The project demonstrates the practical implementation of the following Object-Oriented

Programming concepts:

- Classes and Objects
- Encapsulation
- Constructors
- Methods
- Data Abstraction
- Collections Framework
- Input Validation

These concepts help create a structured, modular, and maintainable software solution.

Gantt Chart

Task	April	May	June
Requirement Analysis	✓		
System Design	✓	✓	✓
Implementation of Candidate class		✓	
Voting & Validation Logic		✓	
Result Generation Module		✓	
Documentation		✓	✓

Conclusion

The Online Voting System was successfully developed using Java and Object-Oriented Programming principles to simulate a simple and efficient electronic voting process. The system allows voters to cast votes, prevents duplicate voting through voter ID validation, and automatically calculates and displays election results.

Through the development of this project, various core Java concepts such as classes, objects, encapsulation, methods, collections, loops, and conditional statements were implemented in a practical environment. The use of ArrayList for candidate management and HashSet for voter verification helped ensure efficient data handling and accurate vote processing.

The project successfully meets its primary objectives of maintaining voting integrity, counting votes accurately, and providing a user-friendly menu-driven interface. In addition to strengthening programming skills, the project provided valuable experience in system design, problem-solving, testing, and software development practices.

Although the current implementation is limited to a console-based application, it establishes a strong foundation for future enhancements such as graphical user interfaces, database integration, secure authentication mechanisms, online voting capabilities, and persistent data storage. Overall, the Online Voting System demonstrates the effective application of Object-Oriented Programming concepts to solve a real-world problem in a structured and efficient manner.

